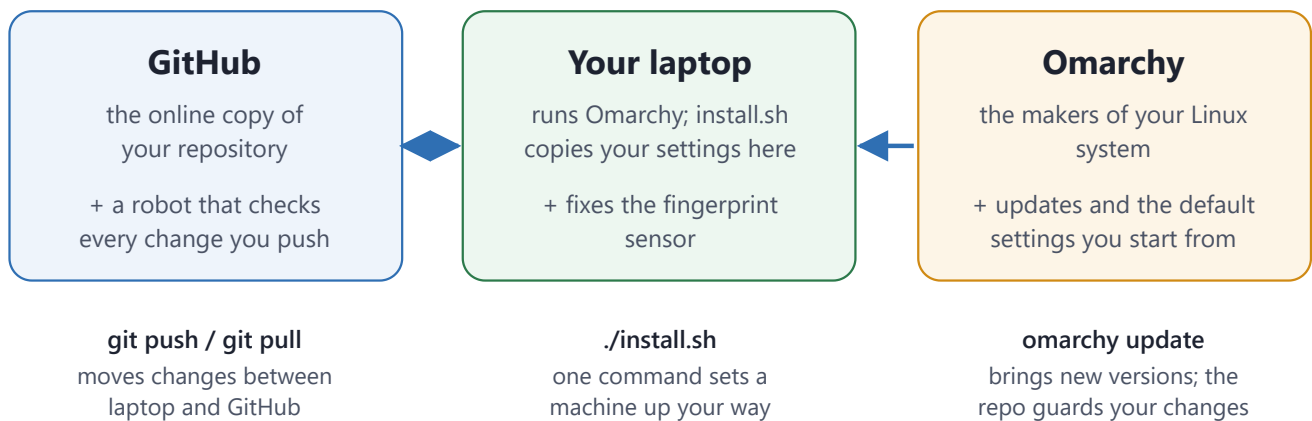


Your Omarchy Setup, Explained

Everything in the **omarchy-quattro-dotfiles** repository — what each file is, what the setup script does, how the fingerprint fix works, what happens automatically, how your settings are protected, and what GitHub checks for you — in everyday words.



Repository: github.com/dev-learner-supreme/omarchy-quattro-dotfiles · Written 26 September 2026, describing the repo as of commit `501fc40`

Main machine: Acer Swift Go 14 (SFG14-71) running Omarchy 4 · Also used on: headless Realme Slimbook home PC

What's in this guide

Read section 1 for the big picture. Everything after that is detail you can dip into. Section 11 is the to-do list for your laptop.

1. **The big picture** — what the repo is for, in one page
2. **Words you'll see** — a plain-English dictionary of every technical term
3. **Every file in the repo** — what it is and what it changes
4. **Your personal settings, in detail** — keyboard shortcuts, touchpad, top bar, terminal, sound...
5. **What the setup script does, step by step** — `install.sh`
6. **How your settings are copied without losing changes** — the "never overwrite silently" rules
7. **The fingerprint story** — the problem, the fix, and where fingerprint works
8. **What happens automatically** — after updates, at start-up, on theme change
9. **The safety nets** — every protection built in, and why
10. **The robot on GitHub** — the automatic checks and that "Actions" page
11. **Your to-do list on the laptop** — the first real run, step by step
12. **Everyday commands** — a cheat sheet
13. **Honest limits and open items** — what isn't perfect, and why
14. **What changed today** — the history of the work, in plain words

1. The big picture

Your repository ("repo") is a folder that lives on GitHub and on your laptop. It holds three kinds of things.

① Your personal settings

The handful of settings files where you changed something from Omarchy's normal setup — your keyboard shortcuts, touchpad feel, top bar layout, terminal font, sound priorities, git identity and more.

② A setup script

`install.sh` — one command that turns a fresh Omarchy install into *your* setup: installs a few apps, copies your settings into place, sets up your fingerprint sensor, and so on.

③ A fingerprint fix

Your laptop's fingerprint sensor doesn't work properly with the standard Linux software. The repo carries a special driver (the software that talks to the sensor), a tool that sets it all up, and protections so updates don't undo it.

④ Automatic checks

A set of tests, plus a "robot" on GitHub that runs them every time you send changes up. If a change would break something, GitHub shows a red X and emails you.

An everyday comparison

Think of moving into a new flat that comes fully furnished (that's Omarchy). The repo is your moving box: it holds only the things you brought that are different from what the flat already has, plus an instruction sheet (`install.sh`) for putting them in place. The fingerprint fix is a special adapter your laptop needs that the flat doesn't provide. The GitHub robot is an inspector who checks the instruction sheet every time you edit it.

The one idea that matters most

Omarchy keeps improving its own default settings through updates. If your repo kept full copies of *every* settings file, each run of the setup script would quietly throw those improvements away. So the repo now keeps **only the files you actually changed**, and the setup script **never overwrites a file that changed on your machine without asking you first**. Everything else stays Omarchy's.

Where things live

Place	What's there
GitHub	The master copy of the repo, its history, and the automatic checks. Address: <code>github.com/dev-learner-supreme/omarchy-quattro-dotfiles</code>
The repo folder on your laptop	Your working copy (for example <code>~/dotfiles</code>). You run <code>./install.sh</code> from here. It must stay in place: the fingerprint tool finds the driver through it.
<code>~/.config</code> on the laptop	Where your settings are actually used. <code>install.sh</code> copies the repo's files into it.

<code>/etc/pam.d</code> on the laptop	System files that decide how you prove it's you (password or fingerprint). Only the fingerprint tool edits these, carefully.
---------------------------------------	--

2. Words you'll see

Every technical word used in the repo and in this guide, in plain terms.

Your computer's software

Word	What it means
Linux / Arch Linux	The operating system (like Windows or macOS). Arch is a popular flavour of Linux that Omarchy is built on.
Omarchy	A ready-made, good-looking setup of Arch Linux by David Heinemeier Hansson (DHH). Version 4 is nicknamed "Quattro".
Hyprland	The part of Omarchy that draws and arranges your windows and handles keyboard shortcuts.
Omarchy shell	The top bar, menus, notifications and lock screen. It's built from small pieces called plugins (the clock, the battery icon, the lock screen...).
Plugin / clone	A plugin is one piece of the shell. A <i>clone</i> is your own copy of one of Omarchy's plugins, so you can change it without editing Omarchy's files.
Terminal	The text window where you type commands. You use Ghostty ; others installed are Alacritty, Kitty and Foot.
Shell (bash / zsh)	The program inside the terminal that understands your commands. You use zsh ; bash is the default one Omarchy also sets up.
Command	A line you type in the terminal, like <code>./install.sh</code> .
Script	A file containing a list of commands that run one after another. <code>install.sh</code> is a script.
Settings file / config / dotfile	A small text file that stores a program's settings. "Dotfile" because many start with a dot (<code>.bashrc</code>), which hides them.
<code>~</code> (home folder)	Your personal folder, <code>/home/arun</code> . <code>~/.config</code> is the folder of settings inside it.
<code>/etc/skel</code>	Omarchy's master copy of its default settings. New users get copied from it; the setup script compares against it to spot untouched defaults.
Update / migration	<code>omarchy update</code> installs new versions. Migrations are small fix-up scripts that run during an update and sometimes edit your settings files (for example to fix a bug or a security problem).
Hook	A script Omarchy runs automatically at a certain moment — after an update, at start-up, when you change theme.
Symlink	A shortcut file that points to another file. The fingerprint command in <code>~/.local/bin</code> is a symlink to the copy in your repo.

Security and passwords

Word	What it means
sudo	"Do this as the administrator." Commands that change the system ask for your password (or fingerprint) first.
polkit / admin pop-up	The window that asks for your password when an app needs admin rights.
PAM	The rulebook Linux follows to decide how you prove it's you — separately for sudo, admin pop-ups, and the lock screen. Its files live in <code>/etc/pam.d</code> .
"sufficient"	The PAM rule used for fingerprint: a matching finger is enough to get in; if it doesn't match (or you skip it), you're asked for your password as usual.
Lid gate	An Omarchy rule that skips the fingerprint step when the laptop lid is closed (you can't reach the sensor), going straight to the password.
Checksum (sha256)	A long "fingerprint" of a file's exact contents. If even one byte changes, the checksum changes — so it proves a file hasn't been corrupted or swapped.
Backup	A copy saved before anything is changed, in <code>~/.dotfiles-backup/<date-time>/</code> .

The fingerprint sensor

Word	What it means
Driver	Software that lets Linux talk to a piece of hardware — here, the fingerprint sensor.
libfprint / fprintd	libfprint is Linux's fingerprint driver collection. fprintd is the background service that uses it.
EgisTec, 1c7a:0584	The maker of your sensor (Egis Technology) and its ID number.
Match-on-Chip	Your fingerprint is stored and checked <i>inside the sensor itself</i> , not on the laptop's disk.
SDCP	A secure "handshake" the laptop and this kind of sensor must do. Without it, the sensor forgets your finger after the first use. The special driver adds it.
Enroll / verify	Enroll = teach the sensor your finger. Verify = test that it recognises it.

Software packages

Word	What it means
Package	A bundled, installable piece of software (like an app from an app store).
pacman	Arch Linux's app installer and updater.
AUR	A community collection of extra packages, not officially checked by Arch.
PKGBUILD	A recipe that tells the computer how to build a package from its source code.
IgnorePkg ("pinning")	A setting in <code>/etc/pacman.conf</code> that tells updates "never replace this package". Used to protect the fingerprint driver.

Git and GitHub

Word	What it means
Repository (repo)	A folder whose full history of changes is tracked.
Git / GitHub	Git tracks the changes; GitHub is the website that stores the repo online.
Commit	A saved snapshot of changes, with a short description.
Push / pull	Push = send your commits up to GitHub. Pull = bring GitHub's latest down to your laptop.
CI / GitHub Actions	GitHub's robot that runs your checks automatically on every push ("Continuous Integration").
ShellCheck	A tool that reads scripts and points out common mistakes.
Test / sandbox / stand-in	A test runs the scripts and checks the result. A sandbox is a fake, throwaway computer set up just for the test. Stand-ins ("stubs") are fake versions of Omarchy's commands that only pretend to act and write down what they were asked to do.

3. Every file in the repo

Grouped by job. **yours** = your personal settings. **tool** = part of the machinery.

The tools (programs)

File	What it is
<code>install.sh</code> tool	The setup script you run. Six steps: apps → settings → fingerprint → optional extras → SSH keys → reload. Safe to run again and again (section 5).
<code>bin/egismoc-fingerprint</code> tool	The fingerprint tool. Installs the special driver, teaches the sensor your finger, turns fingerprint on for sudo / admin pop-ups / lock screen, and keeps it all working (section 7). Linked into <code>~/.local/bin</code> so you can type <code>egismoc-fingerprint</code> anywhere.
<code>lib/ui.sh</code> tool	Shared helpers used by both tools: coloured messages, yes/no questions, and keeping sudo "unlocked" during a long run so you aren't asked for your password repeatedly.
<code>lib/dotfiles.sh</code> tool	The part of the setup that copies your settings into place without ever silently overwriting a changed file, and removes files you deleted from the repo (section 6).

Your settings (13 files)

File	What it changes (details in section 4)
<code>.bashrc</code> yours	Bash shell start-up: SSH key helper, your own programs folder, Android developer tools.

<code>.zshrc</code> yours	Zsh shell (the one you use): history, prompt, shortcuts, suggestions, colours, Android and Google Cloud tools.
<code>.config/ghostty/config</code> yours	Terminal: opens zsh, font size 12 (Omarchy's is 9), and a few shortcuts.
<code>.config/git/config</code> yours	Your name and email on commits, GitHub login through the <code>gh</code> tool, and git preferences.
<code>.config/hypr/bindings.lua</code> yours	Keyboard shortcuts: screenshot, app launcher, F7 and F8 keys.
<code>.config/hypr/hyprland.lua</code> yours	Main window settings file: loads Omarchy's defaults, then your files, plus rules for the Android emulator and virtual machines.
<code>.config/hypr/input.lua</code> yours	Keyboard repeat speed, mouse and touchpad feel, three-finger swipe.
<code>.config/omarchy/shell.json</code> yours	The top bar: what's on it, where, clock format, transparency; screensaver and lock timing.
<code>.config/omarchy/extensions/omarchy-menu.jsonc</code> yours	One change to the Omarchy menu: <i>Setup > Security > Fingerprint</i> runs your fingerprint tool instead of Omarchy's (which would break your sensor).
<code>.config/omarchy/hooks/...</code> yours	Four automatic scripts: after theme changes, before package-list refreshes, after updates, at start-up (section 8).
<code>.config/wireplumber/.../50-alsa-output-priority.conf</code> yours	Sound: headphones beat speakers, speakers beat HDMI monitors.
<code>.local/share/wireplumber/scripts/sof-autoswitch.lua</code> yours	Sound: switches automatically when you plug in or unplug headphones.

The fingerprint driver kit (`packages/`)

File	What it is
<code>libfprint-egismoc-sdcp-git-r1831.4d128d4-1-x86_64.pkg.tar.zst</code>	The ready-built special driver (156 KB). Installed straight from here, so setup works even without internet.
<code>.sha256sums</code>	The driver file's checksum. The tool refuses to install a driver file that doesn't match it.
<code>libfprint-egismoc-sdcp/PKGBUILD</code>	The recipe to rebuild the driver from its source code if the ready-built copy is missing. Locked to one exact, known-good version.
<code>libfprint-egismoc-sdcp/README.md</code>	Explains the driver, how it's installed, and how to move to a newer version on purpose.

Tests and the GitHub robot

File	What it is
<code>tests/run.sh</code>	Runs all 72 checks in about 10 seconds. You can run it yourself any time.

<code>tests/lib/sandbox.sh</code>	Builds the fake, throwaway computer the tests run on: fake home folder, fake system files, fake fingerprint sensor, and stand-ins for Omarchy's commands. The real system is never touched.
<code>tests/fixtures/...</code>	A tiny made-up imitation of Omarchy's lock screen for the tests to work on.
<code>.github/workflows/ci.yml</code>	Instructions for the GitHub robot: what to check on every push (section 10).

Documents

File	What it is
<code>README.md</code>	The repo's front page on GitHub: overview, shortcuts, sound rules, how to install, the settings-copy rules, tests, safety features.
<code>SETUP_AND_ARCHITECTURE.md</code>	A deeper technical guide: packages, the fingerprint setup and recovery, the SSH setup, the file layout, update safety.
<code>SYSTEM_HEALTH_AND_AUTH_AUDIT.md</code>	An earlier health check of your login and fingerprint setup: why a harmless "exit code 1" message kept appearing in the system log (the lid gate, fixed with <code>quiet_log</code>), how sudo, the lock screen and admin pop-ups are configured, why the lock screen needs the 1.5-second delay, a comparison with Omarchy's own setup, and safe read-only commands for diagnosing problems.
<code>SSH_SETUP_GUIDE.md</code>	How to create an SSH key (a secure login key for GitHub and servers), add it to GitHub, and have the system remember its passphrase once per login.
<code>CLAUDE.md</code> (and <code>AGENTS.md</code>)	Instructions for AI assistants (like Claude Code) working on this repo: the layout, the hard rules (never run Omarchy's own fingerprint setup here, never hand-edit the lock-screen copy, only keep changed files...), and the checks to run. <code>AGENTS.md</code> is the same file under the name other AI tools look for.
<code>.gitignore</code>	Tells git which files never to save: temporary files, logs, backups, and leftovers from building the driver by hand.

Deliberately NOT in the repo

These were removed today because they were just Omarchy's own defaults (unchanged, or an old version). Keeping them would have undone Omarchy's future improvements every time you ran the setup:

Alacritty, btop, Foot, Kitty, lazygit and Starship settings; `autostart.lua`, `looknfeel.lua`, `monitors.lua`, `hyprsunset.conf` (night light), `xdph.conf` (screen sharing); the Bluetooth audio file; the `hyprmoncfg` monitor layout; the three custom themes (awsm-changi, luminous, sora-koi).

The old **Kitty** file mattered most: it switched back on a setting (`allow_remote_control yes`) that lets any program on your laptop control your Kitty windows — something a recent Omarchy security update had switched off.

4. Your personal settings, in detail

Keyboard shortcuts — `bindings.lua`

Press	What happens	Instead of Omarchy's
Super + Shift + S	Take a screenshot	The Print Screen key (now does nothing)
Alt + Space	Open the app launcher	Super + Alt + Space
F7 · Super + P · Display key	Open the hyprmoncfg monitor arranger (only if hyprmoncfg is installed)	Omarchy's Super + P action
F8 · Super + L	Lock the screen	Omarchy's Super + L action

"Super" is the Windows-logo key. F7 and F8 are the Acer's display and lock keys.

Touchpad, mouse and keyboard — `input.lua`

- **Key repeat:** holding a key starts repeating after 0.25 s, at 40 characters per second (faster than normal).
- **Pointer speed:** a bit faster (sensitivity 0.35), with no acceleration ("flat") so movement is predictable.
- **Touchpad:** natural scrolling (content moves with your fingers, like a phone), slower scrolling (0.4), two-finger click = right-click, and the touchpad stays active while you type.
- **Gesture:** swipe sideways with three fingers to switch workspaces (virtual desktops).

Windows — `hyprland.lua`

Loads Omarchy's standard setup and then your other files. Your addition: the **Android emulator** and **QEMU virtual machines** open as floating windows (not squeezed into tiles) at full opacity (not see-through).

The top bar — `shell.json`

Part	Setting
Look	At the top of the screen, transparent background.
Left	Omarchy menu button · workspaces · a screen-time widget (<code>agx.screen-time</code>)
Middle	Status indicators · the clock · keyboard layout · weather · update notifier
Right	System tray · hyprmoncfg · AI agents usage · network · a Bluetooth audio widget (<code>ssupt.bluetooth-audio</code>) · volume · monitor · battery with percentage shown
Clock	Shows e.g. "Sat 26 Sep 10:20"; alternate view "26 September W39 2026". Also stores a birth year (1999) and life expectancy (85), which the clock appears to use for a life-progress display.
Idle	Screensaver after 2½ minutes without use, lock after 5 minutes (Omarchy's usual timings).

Note

Three items on the bar are add-ons from other people (screen time, hyprmoncfg, Bluetooth audio). `install.sh` does not install them, so on a brand-new machine those spots stay empty until you add them through *Setup > Plugins > Add* in the Omarchy menu.

Terminal — Ghostty `config`

- Opens **zsh** instead of bash; font JetBrainsMono Nerd Font at size **12**; colours follow your Omarchy theme.
- Comfortable spacing around text, no "are you sure?" when closing, solid non-blinking cursor.
- Shift+Insert pastes, Ctrl+Insert copies; Shift+Enter and Alt+Shift+Enter send special codes so terminal apps (like tmux) can tell them apart.
- Super+Ctrl+Shift+Alt+arrow keys resize split panes; slightly slower mouse scrolling; a speed fix for Hyprland.

Shells — `.zshrc` and `.bashrc`

- **History:** remembers your last 10,000 commands, shared between open windows, without duplicates; commands you start with a space aren't saved.
- **Prompt:** the Starship prompt, matching Omarchy's look. **Tab completion** is switched on.
- **Shortcuts:** `..` goes up one folder (`...` two, and so on); `-` goes back to the previous folder; `ls`, `l`, `ll`, `la` list files with icons; `lt` shows a folder tree.
- **Autosuggestions:** grey "ghost" text suggests a command from your history as you type (press → to accept). **Syntax highlighting** colours commands as you type them.
- **Developer tools:** the Android SDK tools (emulator, adb) and, if present in `~/Downloads`, the Google Cloud command-line tools.
- **SSH:** both shells know where the SSH key helper lives (see section 5, step 5). `.bashrc` also loads Omarchy's standard bash setup and adds `~/local/bin` (your own programs) to the front of the search path.

Git — `git/config`

- Commits are signed with your name, **Arun Rao Balappa**, and email **arunraobalappa@gmail.com**.
- Logging in to GitHub is handled by the `gh` tool, so you aren't asked for passwords.
- Short commands: `git co` (checkout), `git br` (branch), `git ci` (commit), `git st` (status).
- Tidier behaviour: new repos start on `master`; pulling replays your changes on top instead of making merge knots; clearer change views; newest branches listed first; git remembers how you solved a conflict and reuses it.

Sound — the two WirePlumber files

- **Priority:** Headphones (1500) > laptop speakers (1200) > HDMI monitors (600). Plugging in a monitor won't steal your sound; plugging in headphones will take it.
- **Auto-switch:** the laptop's sound chip (Intel SOF with a Realtek ALC256) needs a nudge to change between headphones and speakers. The small script does that automatically when you plug in or unplug.

5. What the setup script does, step by step

You run it from the repo folder with `./install.sh`. Wherever Omarchy has its own tool for a job, the script uses it, so the result matches what the Omarchy menu would do.

Before it starts

- Refuses to run as the administrator ("don't run me with sudo") — it asks for your password itself when needed.
- Checks this really is Omarchy, and warns if the version is older than 4.
- Makes sure only one copy runs at a time. If an earlier run was killed, it notices and cleans up.
- Asks for your password once, then keeps it "unlocked" in the background so you aren't asked again mid-way.

1 Apps

Installs, if missing: **omarchy-zsh** (zsh plus the Starship prompt and handy tools: eza, zoxide, fzf, bat, fd, mise, syntax highlighting), **zsh-autosuggestions**, **usbutils** (lists USB devices), **restic** (the engine behind Omarchy's backup feature) and **rclone** (connects to cloud storage such as **Google Drive**). Then makes sure **Ghostty** is installed and is your default terminal.

2 Your settings

Copies the repo's 13 settings files into place — carefully. It never replaces a file that changed on your machine without asking, and it removes files you've deleted from the repo. The exact rules are in section 6. It also links the fingerprint tool into `~/.local/bin`.

This happens *before* the fingerprint step on purpose: your stored top-bar file would otherwise switch off the lock-screen copy that step 3 turns on.

3 Fingerprint

- **EgisTec sensor** (your laptop): runs `egismoc-fingerprint setup` (section 7).
- **Any other sensor**: offers Omarchy's own fingerprint wizard.
- **No sensor** (like your home PC): skips.
- If fingerprint setup fails, the rest of the setup still carries on, and you can re-run it later.

4 Optional extras

Asks whether to install **hyprmoncfg** (a monitor arranger for the F7 key) and the **Brave Origin** browser. Both default to "no". Needs the AUR (the community collection) to be reachable.

5 SSH key helper

An SSH key is a secure key file you use to log in to GitHub and servers. This step switches on the system's key helper ("ssh-agent"), tells every app where to find it, and sets your SSH settings (`~/.ssh/config`) so your key (`id_ed25519`) is remembered after you type its passphrase once per login.

6 Reload

Restarts the sound system so the sound rules apply; reloads Hyprland and reports any errors in your window settings; refreshes open terminals; restarts the top bar (unless the screen is locked); tells you where any backups went; prints "Setup complete!".

Two ways to run it

- `./install.sh` — normal. Asks you questions when a decision is needed.
- `./install.sh -y` — unattended. Every question takes its default answer: optional extras are skipped, files changed on your machine are kept (and listed at the end), and fingerprint enrolment is left for later because it needs your finger.

6. How your settings are copied without losing changes

Your settings files can change on the laptop without you doing anything: Omarchy's updates edit some of them, and apps like `hyprmoncfg` add lines. So the setup script keeps a small record of exactly what it copied last time (in `~/.local/state/omarchy-dotfiles/deployed`) and handles each file like this:

If the file on your laptop is...	...then the script
missing	Adds it.
identical to the repo's	Does nothing.
exactly what it copied last time	Updates it to the repo's newer version (nobody else touched it, so it's safe).
Omarchy's untouched default	Replaces it with yours, keeping a backup. (It checks against Omarchy's master copy in <code>/etc/skel</code> .)
changed since the last copy	Stops and asks you (see below). In <code>-y</code> mode it keeps yours and lists it at the end.
deleted from the repo, unchanged here	Removes it, keeping a backup.
deleted from the repo, but changed here	Asks (default: keep it). Either way it stops managing that file.
a shortcut (symlink) to somewhere else	Leaves it alone — another tool is managing it.

When it asks you about a changed file

Choice	What happens
Keep mine	Nothing changes. It will ask again next time, because the laptop and the repo still differ.
Use the repo's version	Replaces the laptop's file with the repo's. Your version is saved as a backup.

Copy mine into the repo	Copies the laptop's version into the repo folder. Then commit and push it (section 12) so it's saved on GitHub. No more questions about that file.
Show the difference	Shows the lines that differ (lines marked <code>-</code> are the repo's, <code>+</code> are the laptop's), then asks again.

Why this matters

Before today, the script silently replaced these files every time, only saving the old one in a backup folder nobody looked at. That's how an Omarchy security fix to your Kitty settings kept getting undone. Now nothing you (or Omarchy) changed is replaced without you deciding.

Adding a new setting to the repo

If you customise another file in the future (say your Alacritty settings), copy it from `~/.config/...` into the same place inside the repo folder, then commit and push. Only add files you've actually changed.

Backups

Every replaced or removed file is saved first in `~/.dotfiles-backup/<date>_<time>--<number>/`, under the same path it had in your home folder. The number makes each run's folder unique, so a second run can never overwrite the first run's backups.

7. The fingerprint story

The problem

Your sensor (EgisTec, ID `1c7a:0584`) is a "Match-on-Chip" sensor: it keeps your fingerprint inside itself. To use it, the laptop has to do a secure handshake with it (called **SDCP**). The standard Linux fingerprint driver — and Omarchy's own version of it — don't do that handshake for this sensor. The result: you can teach it your finger, and it looks like it worked, but the very first time you use it, the finger is "forgotten".

The official fix is still waiting to be accepted by the Linux fingerprint project (as of September 2026). Until it lands and reaches Omarchy, you need a special driver.

The fix

- **The special driver** is a community version of the Linux fingerprint driver, written by a developer known as TenSeventy7, that adds the missing handshake. It includes support tuned specifically for your sensor, and other people have confirmed it working on the same sensor in the same Acer model family.
- **Locked to one exact version** (July 2025, code `4d128d4`), so it can't change underneath you.
- **A ready-built copy** lives in the repo, checked against its checksum before every install. If it's missing, the tool rebuilds it from the locked recipe — never from the unchecked community package collection.

What `egismoc-fingerprint setup` does

1. Checks the sensor is really your EgisTec one. If it's a different sensor, it hands over to Omarchy's own wizard instead.
2. Installs the special driver (replacing the standard one in a single safe step), and pins it so updates won't replace it.

3. Installs `fprintd` (the fingerprint background service) and restarts it.
4. If fingerprints were ever saved for the administrator account ("root"), offers to delete them — they would block saving yours.
5. Asks you to place your **right index finger** on the sensor repeatedly until it's learned, then asks you to **verify** it. This is exactly the step that fails without the special driver.
6. **Only after the finger is saved and verified**, turns fingerprint on for sudo, admin pop-ups and the lock screen (same order Omarchy uses, so login never points at a sensor that can't match).
7. Checks that sudo still works with the new settings (by asking you to authenticate again), and offers to undo everything if not.
8. Sets up your lock-screen copy with the 1.5-second delay (below).

Where your fingerprint works

Place	How it behaves
sudo in the terminal	Touch the sensor, or type your password. Press Ctrl+C at the fingerprint prompt to go straight to the password.
Admin pop-ups	Same: finger or password.
Lock screen (Super+Ctrl+L, F8, Super+L)	Waits for your finger with no time limit; the password box works at the same time.
Lid closed (docked)	Fingerprint is skipped automatically; you go straight to the password.
The login screen after start-up	Untouched — always your password. This is also your guaranteed way back in.

The lock screen copy and its 1.5-second delay

After a failed or timed-out attempt, Omarchy's lock screen restarts the fingerprint reader after 0.25 seconds. Your sensor needs about 1.5 seconds to reset; any faster and the driver crashes in a loop. So the tool makes **your own copy** of Omarchy's lock screen (using Omarchy's official "clone" feature) and changes that one number.

A copy doesn't receive Omarchy's future lock-screen updates by itself, and the lock screen is a security feature. So **after every Omarchy update and at every start-up**, your copy is compared with Omarchy's current lock screen. If Omarchy changed it, your copy is rebuilt from the new version, the delay is re-applied, and you get a "Lock screen updated" notification.

- It never swaps the lock screen while your screen is locked — it waits for next time.
- If a rebuild fails, you're left on Omarchy's normal lock screen (still secure; fingerprint may be unreliable) and get a notification. It tries again next time by itself.
- If Omarchy changes its code so the delay can't be applied any more, you get a notification rather than a silent failure.

What protects the fix from being undone

- **Pinned driver:** updates are told never to replace it, and a hook re-adds that instruction whenever Omarchy resets its package settings.
- **Menu override:** *Setup > Security > Fingerprint* in the Omarchy menu runs your tool, not Omarchy's.

- **Watchdog:** if the driver ever gets replaced anyway, you get a clickable "Fingerprint driver replaced" notification after the next update or start-up. Clicking it runs the setup again.
- **Repairs after updates:** if an update changes the lock-screen timing setting or the lid-gate setting, the post-update hook puts it back.

Never do these on the laptop

- Don't run `omarchy setup security fingerprint` from a terminal (the menu entry is already safe).
- Don't install `libfprint` or `libfprint-git`.

Either one replaces the special driver, and fingerprint stops working. If it happens: run `egismoc-fingerprint setup`.

The fingerprint tool's commands

Command	What it does
<code>egismoc-fingerprint setup</code>	The full setup above. Safe to repeat.
<code>egismoc-fingerprint status</code>	A read-only health report. Each line shows ok or !! for: the sensor, the driver, the pin, your saved finger, sudo, admin pop-ups, the lock screen's setting, and the lock-screen copy.
<code>egismoc-fingerprint check</code>	What the automatic hooks run: no password needed, no questions. Sends a notification if the driver was replaced, and keeps the lock-screen copy current.
<code>egismoc-fingerprint detect</code>	Prints your sensor's ID if it's an EgisTec one (used by <code>install.sh</code>).

8. What happens automatically

"Hooks" are scripts Omarchy runs by itself at certain moments. Yours all do nothing on a machine without the special fingerprint driver (like your home PC).

When	Hook	What it does
You change theme	<code>theme-set</code>	Passes the new theme's name to a tool called <code>omazed</code> . (Going by its name it syncs the theme to the Zed code editor; this wasn't checked.)
Omarchy resets its package settings (<code>omarchy refresh pacman</code> , or switching update channel)	<code>pre-refresh-pacman</code>	Re-adds the "never replace the fingerprint driver" instruction before any packages are updated.
At the end of <code>omarchy update</code>	<code>post-update</code>	Re-checks the driver pin; repairs the lock screen's fingerprint setting and the lid gate if an update changed them; checks the driver is still there (notification if not); rebuilds the lock-screen copy if Omarchy updated its lock screen.

Every start-up	<code>post-boot</code>	The same driver and lock-screen checks, without anything that needs a password.
----------------	------------------------	---

You may be asked for your password during an update

Omarchy deliberately "locks" admin rights again before running hooks. If the post-update hook finds something to repair in the system files, it has to ask for your password (or finger) — but only then. If nothing needs fixing, it asks nothing.

Notifications you might see

Notification	Meaning and what to do
Fingerprint driver replaced	Something swapped out the special driver. Click it (or run <code>egismoc-fingerprint setup</code>).
Lock screen updated	Omarchy improved its lock screen; your copy was rebuilt with the fix. Nothing to do.
Lock screen refresh failed	You're on Omarchy's normal lock screen for now. It retries next update/start-up; or click to re-run setup.
Lock screen fingerprint fix not applied	Omarchy changed its lock-screen code; the 1.5 s delay no longer fits. The repo needs an update — ask for help.
Editing Cloned Plugin	Omarchy's own message whenever a copy of a plugin is made. Harmless; appears alongside "Lock screen updated".

9. The safety nets

Anything that touches passwords or the system is treated as high-risk. These protections are built in — and the tests (section 10) prove each one works.

Protection	What it guards against
Password always works	Fingerprint is only ever an extra way in. The script refuses to keep any change that would remove the password route.
Login screen untouched	Even if sudo broke, you could still log in normally.
Backup before every system edit	The three login rulebook files are copied before they're touched.
Automatic undo	If editing them fails part-way, or you press Ctrl+C, all three are put back exactly as they were. A later, unrelated failure doesn't undo a finished fingerprint setup.
Strict change check	After editing, the only allowed differences are the fingerprint lines. Anything else changed → everything is restored.
Real sudo re-check	Asks you to authenticate through the new settings while keeping your current admin session open, so a broken setup can still be undone. If it fails, it offers a one-step restore.
Enroll before switching on	Fingerprint login is only switched on after your finger is saved <i>and</i> verified.

Driver checksum	A corrupted or swapped driver file is refused, with a warning.
Locked driver version	The driver can't change underneath you; rebuilding uses the locked recipe, never the unchecked community collection.
No silent overwrites	Settings changed on the laptop are never replaced without asking.
Unique backup folders	Two runs can never share (and wipe) each other's backups.
One run at a time	Two copies of the setup can't collide; a leftover lock from a crashed run is cleared safely.
Lock screen never swapped while locked	Rebuilding the lock-screen copy waits until you're unlocked.
Failure doesn't cascade	If fingerprint setup fails, your settings, SSH and the rest still get set up.

10. The robot on GitHub

Every time you push changes, GitHub borrows a brand-new Linux computer in its cloud, runs your checks on it, reports the result, and throws the computer away. It never touches your laptop. The instructions are in `.github/workflows/ci.yml`.

What it checks, in order (about 15 seconds in total)

1. **Downloads your repo** onto the fresh computer.
2. **Script grammar** — every script can at least be read without a typo stopping it.
3. **ShellCheck** — scans all scripts for common bugs.
4. **Driver checksum** — the saved fingerprint driver still matches its recorded checksum.
5. **Settings files are valid** — `shell.json` and the menu file are correctly formatted (a mistake there could break your top bar or menu).
6. **The 72 tests** — full simulated runs of the setup and the fingerprint tool on a fake computer.

What the 72 tests cover

Fingerprint and login

- A full unattended install on your laptop's hardware
- Running it again changes nothing
- Other sensors go to Omarchy's wizard; hooks leave other machines alone
- A tampered driver is refused
- A bad or failed login-file edit is undone exactly
- First-time setup saves your finger before switching login on
- A failed sudo re-check offers a restore
- A replaced driver raises the notification

Lock screen and settings

- Lock-screen copy rebuilt after an Omarchy update
- ...but not while locked; recovers after a failed rebuild; warns if the code changed
- New machine: defaults replaced, missing files added
- Repo updates applied without questions
- Changed files kept, replaced, or copied into the repo on request; differences shown
- Deleted files removed (unless changed); shortcuts left alone

To make sure the tests really catch problems, five key protections were deliberately broken one at a time; every break was caught.

Reading the "Actions" page on GitHub

You see	It means
← CI	The name of the checks. Click to see every run (one per push).
The big title, e.g. "ci: run ShellCheck..." #1	The description of the commit you pushed; #1 = the first run ever.
Triggered via push · 501fc40 · master	What started it: your push of that commit to the main branch.
Status: Success · Total duration 15s	Everything passed, in 15 seconds.
Artifacts: –	Runs can save output files for download; yours don't need to.
ci.yml · on: push · "checks" box	The instructions file, its trigger (any push), and its one job. Click "checks" to see each step and its full output.
Usage (left side)	Computer time used. Free for public repos; private ones get a monthly free allowance — 15 seconds per push is tiny.
Re-run all jobs	Runs the same checks again (rarely needed).
Green ✓ / red X next to a commit	Passed / failed. On a failure GitHub emails you; click "checks", then the red step; the output names what failed, e.g. <code>FAIL: sudo restored exactly</code> .

The two notes at the bottom of that page

- ⚠ "Node.js 20 is deprecated" — the first step (downloading your code, `actions/checkout@v4`) is built on an older engine GitHub is retiring. For now GitHub runs it on the new engine anyway, so it's only a warning. Fix: a one-line change to `actions/checkout@v7` (the current version, which uses the new engine).
- 📢 "ubuntu-latest will migrate to Ubuntu 26 beginning October 19, 2026" — the robot's computer will get a newer version of Ubuntu Linux. Your checks only use basic tools, and the instructions install ShellCheck themselves if it's missing. If anything breaks, you'll get the red X and an email.

11. Your to-do list on the laptop

Everything above has been tested on a simulated computer, but the new version hasn't yet run on your real laptop. Do this once, calmly, at a time when you're not in a hurry.

1. **Get the latest version of the repo** — in a terminal, go to your repo folder and pull:

```
cd ~/dotfiles      # or wherever your repo folder is
git pull
```

2. **Run the setup normally** (not `-y`) so it can ask you things:

```
./install.sh
```

3. **Answer any "changed on this machine" questions** (section 6). It will probably ask about `hyprland.lua`, because `hyprmoncfg` added a line to it on your laptop. Choose **"Keep mine"** — that line is specific to this laptop. (It will ask again on future runs; that's expected.) If unsure about any file, choose "Show the difference" first, then "Keep mine".

4. **Keep that terminal window open**, open a **new** terminal, and check `sudo` still works:

```
sudo -k true
```

Touch the sensor or type your password. Only close the first window once this works.

5. **Check the fingerprint health report:**

```
egismoc-fingerprint status
```

Every line should say **ok**.

6. **Check the Kitty security setting** left over from the old setup:

```
grep allow_remote_control ~/.config/kitty/kitty.conf
```

If it says `allow_remote_control yes`, reset Kitty to Omarchy's safe default (a backup is made):

```
omarchy refresh config kitty/kitty.conf
```

7. **Lock the screen** (F8) and unlock with your finger to confirm.

Good to know

- The files removed from the repo (old defaults, themes, the `hyprmoncfg` layout) are **not** deleted from your laptop — the script only removes files it knows it put there. They simply become Omarchy's (or `hyprmoncfg`'s) again. If you want the three themes gone, remove them through the Omarchy menu, but not while one of them is your active theme.
- If you ever reinstall without `hyprmoncfg`, the F7 key and the `hyprmoncfg` bar spot simply do nothing until you install it.

12. Everyday commands

Type these in a terminal. Anything after a `#` is just a note.

Setup

```
./install.sh          # set up / refresh this machine (from the repo folder)
./install.sh -y        # same, without questions (keeps changed files)
./install.sh -h        # show the options
```

Fingerprint

```
egismoc-fingerprint status  # health report: every line should say ok
egismoc-fingerprint setup   # set up again / repair
fprind-verify               # test your finger
sudo -k true                 # test sudo with a fresh authentication
```

Saving a settings change to GitHub

```
git status          # what changed in the repo folder
git diff            # the exact lines that changed
git add .config/hypr/bindings.lua    # choose a file to save
git commit -m "Change screenshot key" # save a snapshot with a note
git push            # send it to GitHub (the robot then checks it)
```

Checking your work before pushing

```
tests/run.sh        # all 72 checks, about 10 seconds
tests/run.sh dotfiles # only tests with "dotfiles" in their name
```

Useful Omarchy commands

```
omarchy update      # update everything (hooks run at the end)
omarchy version      # which Omarchy you're on
omarchy restart shell # restart the top bar
omarchy refresh config <file> # reset one settings file to Omarchy's default
omarchy plugin list  # see the top-bar/shell pieces and which are on
```

If something goes wrong

Problem	What to do
Fingerprint stopped working	<code>egismoc-fingerprint status</code> , then <code>egismoc-fingerprint setup</code> . Your password always still works.
sudo won't accept anything	Don't close your open terminal. The pre-edit copies are in the newest <code>~/.dotfiles-backup/.../pam/</code> folder; the setup's final check offers to restore them. As a last resort, reboot and pick the snapshot from before the change in the boot menu (Omarchy takes one before updates).
A settings file looks wrong	The previous version is in <code>~/.dotfiles-backup/</code> ; or <code>omarchy refresh config <file></code> for Omarchy's default.
Red X on GitHub	Open the run, click "checks", then the red step; the output names what failed.

13. Honest limits and open items

Limits to be aware of

- **Not yet run on the real laptop.** Everything was tested on a simulated computer. The first real run (section 11) is the true test.
- **Community driver.** The special driver hasn't been reviewed by the official Linux fingerprint project. That's fine for a personal laptop, but it isn't as thoroughly checked as official software.
- **No automatic driver updates.** Because it's locked to one version, it won't get fixes on its own. Moving to a newer version is a deliberate step (described in `packages/libfprint-egismoc-sdcp/README.md`). When the official fix reaches Omarchy, the whole special-driver setup can be retired.

- **Fingerprint is weaker than a password.** Anyone who can press your finger on the sensor (while you're asleep, for example) gets sudo. You chose to keep fingerprint for sudo, with the password as the backup.
- **The checksum catches accidents, not attackers.** Someone with access to your GitHub account could change both the driver and its checksum. Keep two-factor login switched on for GitHub.
- **The lock-screen fix depends on Omarchy's code.** If Omarchy renames the part the 1.5 s delay changes, you'll get a notification and the repo will need updating.
- **The driver file is stored in git.** Unusual (repos normally hold text), but it's small (156 KB) and makes offline setup possible.
- **Google Cloud lines in .zshrc** point at `/home/arun/Downloads/google-cloud-sdk`. They only do something if that folder exists, so they're harmless elsewhere.

Open items (optional improvements)

- **Clear the GitHub warning** by switching to `actions/checkout@v7` (a one-line change).
- **The repeated `hyprland.lua` question.** Because `hyprmoncfg` adds a laptop-only line, the setup will ask about that file on every run. The script could learn to accept that one line.
- **Bar add-ons.** The three add-on widgets on your top bar aren't installed by the setup; that could be added.
- **hyprmoncfg leftovers.** The F7 shortcut and bar spot stay in your settings even without `hyprmoncfg`; they could be made to appear only when it's installed.
- **Omarchy's own dotfiles feature.** Omarchy's developers are planning an `omarchy dots` command to snapshot your settings around every update and sync them between machines (not released yet). Your setup is compatible with it; when it arrives it could take over the settings-copying half of `install.sh`.

14. What changed today

All the work from 26 September 2026, in the order it was done. Each item is one or more commits on GitHub.

Change	In plain words
Driver recipe	Added the locked recipe for the fingerprint driver, so the repo no longer depends on a stranger's package in the community collection.
First safety fixes	The automatic undo for login-file edits never actually ran (a technical setting was missing) — fixed. The final sudo check didn't really test anything — now it does. The driver file was checked only <i>after</i> installing — now before.
Omarchy-native rebuild	Discovered that Omarchy's own fingerprint menu item would break your sensor, so the menu now runs your tool. Moved the fingerprint logic into its own command; used Omarchy's own tools for the lock-screen copy and terminal; added the "driver replaced" notification; fixed a bug that could make the setup hang for a minute at the end when its output was saved to a file; hooks stopped pinning the driver on your home PC.
Guide for AI assistants	Added <code>CLAUDE.md</code> / <code>AGENTS.md</code> .
Removed <code>hyprmoncfg</code> layout and themes	At your request: <code>hyprmoncfg</code> now creates its own layout when you install it.

Lock-screen auto-refresh	Your lock-screen copy now keeps up with Omarchy's updates (section 7). Also a stricter check that only fingerprint lines change in the login files. sudo keeps fingerprint with the password as backup.
Only your files	Removed 12 files that were just Omarchy's defaults — including the old Kitty file that kept undoing a security fix. Replaced a hard-coded <code>/home/arun</code> in <code>.bashrc</code> .
No silent overwrites	The new settings-copy rules (section 6), and file removal. Fixed a bug where two runs in the same second could wipe each other's backups.
Tests and the GitHub robot	72 automatic checks, run on every push. The first run passed.

In one sentence

Your repo is now a small, tidy set of only your own settings, a setup script that never overwrites your changes without asking, a fingerprint fix that protects itself from updates and keeps your lock screen current, and an automatic inspector that checks every change you make.